

■ AI LeetCode Guide

*An AI-Powered LeetCode Learning Assistant
Built with n8n · SERPER API · OpenRouter (DeepSeek V3)*

Project Documentation
Author: Anuj

Technology	Role
n8n	Workflow Automation
SERPER API	Problem Discovery
OpenRouter	LLM Gateway
DeepSeek V3	AI Note Generation
Markdown	Exported Study Notes

■ Table of Contents

1. Problem Statement
2. Objectives
3. System Architecture
4. Workflow
5. Tech Stack
6. Screenshots & Outputs
7. Sample Output
8. Features
9. Future Enhancements

■ 1. Problem Statement

Preparing for coding interviews often requires solving hundreds of LeetCode problems. While solutions and explanations are available online, candidates frequently spend significant time searching for problem details, understanding approaches, analysing complexities, and creating study notes.

The process is repetitive and time-consuming, especially when revisiting previously solved problems for revision.

To address this, an automated solution is needed that can:

- Accept a LeetCode problem number as input
- Automatically identify the corresponding LeetCode problem
- Generate structured study notes including: Problem Statement, ELI10 Explanation, Intuition, Brute Force Approach, Optimal Approach, Complexity Analysis, Python Solution, and a Dry Run Example
- Export the generated content into a Markdown file for future reference

■ 2. Objectives

- Automate LeetCode problem discovery via search engine integration
- Generate AI-powered structured explanations and solutions
- Reduce time spent manually creating revision notes
- Improve overall interview preparation efficiency
- Store generated notes in portable Markdown format

■■ 3. System Architecture

Step	Component	Description
1	User Input	LeetCode Problem Number
2	SERPER API	Web search for problem details
3	Problem Discovery	Extracts title & description
4	OpenRouter	Routes to DeepSeek V3 model
5	AI Note Generator	Produces structured study notes
6	Markdown Export	Saves .md file to disk

```
User Input (Problem Number)
■
▼
SERPER API ■■■ LeetCode Problem Discovery
```

```
■
▼
OpenRouter / DeepSeek V3
■
▼
AI Study Note Generator
■
▼
Markdown File Export (.md)
```

■ ■ 4. Workflow

Step 1: User enters a LeetCode problem number into the n8n workflow trigger.

Step 2: The Edit Fields node sets the problem number as a workflow variable.

Step 3: An HTTP Request node calls the SERPER API (Google Search) to locate the correct LeetCode problem title and URL.

Step 4: A JavaScript Code node parses the SERPER response and extracts the problem title.

Step 5: A second HTTP Request node sends the problem title to OpenRouter, calling DeepSeek V3 with a structured prompt.

Step 6: A second JavaScript node processes the AI response and formats it into structured Markdown sections.

Step 7: The Convert to File node exports the content as a .md file (e.g. leetcode_1.md).

■ ■ 5. Tech Stack

Technology	Version / Type	Purpose
n8n	Self-hosted	Visual workflow automation platform
SERPER API	REST API	Google Search integration for problem discovery
OpenRouter	API Gateway	Unified LLM gateway supporting multiple models
DeepSeek V3	LLM	AI model for generating structured study notes
JavaScript	Node.js	In-workflow data transformation and parsing
Markdown	.md files	Human-readable exported study notes format

■ 6. Screenshots & Outputs

6.1 n8n Workflow Canvas

The complete n8n workflow consists of seven nodes arranged in a linear pipeline. Each node is connected via success paths, and all nodes show a green checkmark indicating successful execution.



Figure 1 – n8n workflow: Execute Trigger → Edit Fields → HTTP Request (SERPER) → Code → HTTP Request (OpenRouter) → Code → Convert to File

6.2 SERPER API Output

When problem number 1 is submitted, the SERPER API executes a Google search for 'LeetCode 1' and returns structured JSON. The top organic result correctly identifies the problem as 'Two Sum – LeetCode' along with its canonical URL and a snippet describing the problem constraints.

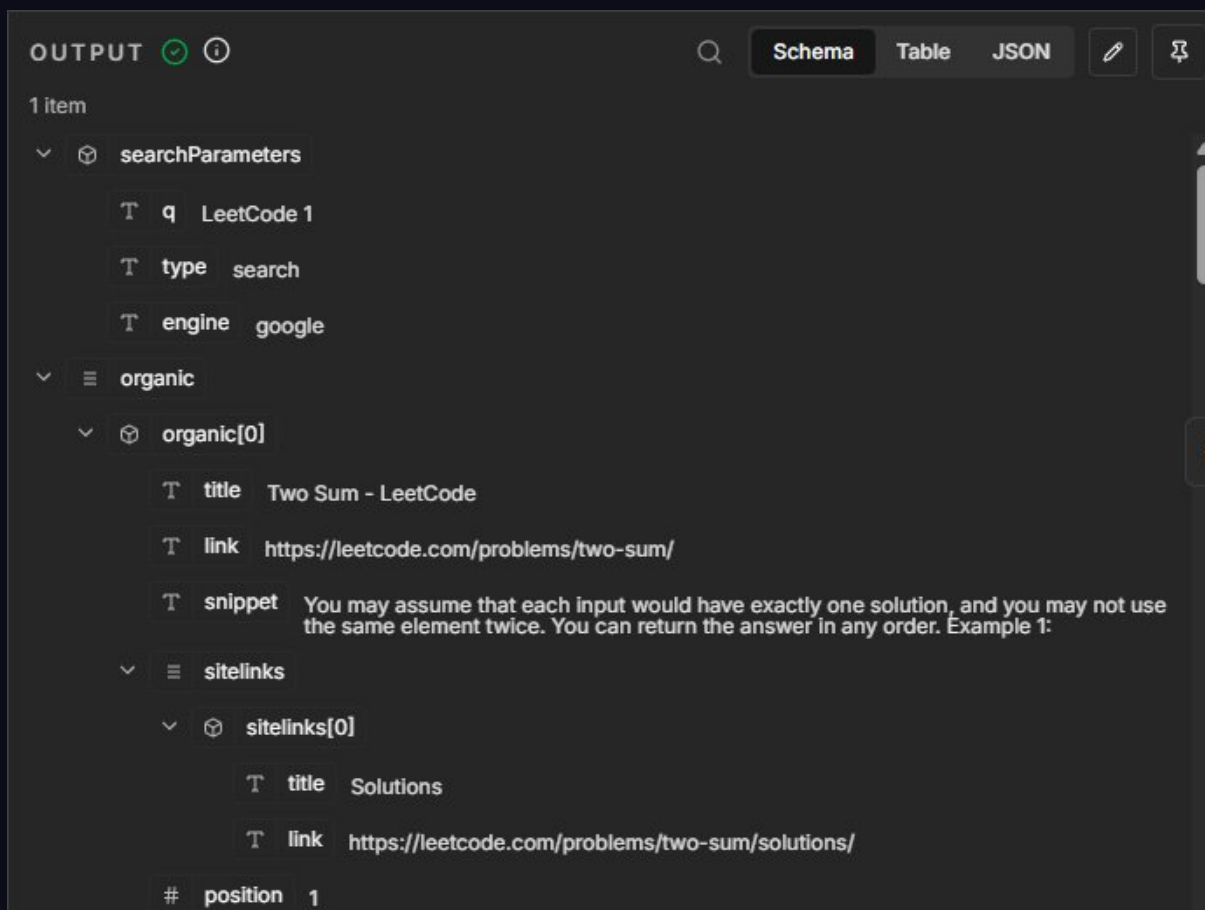
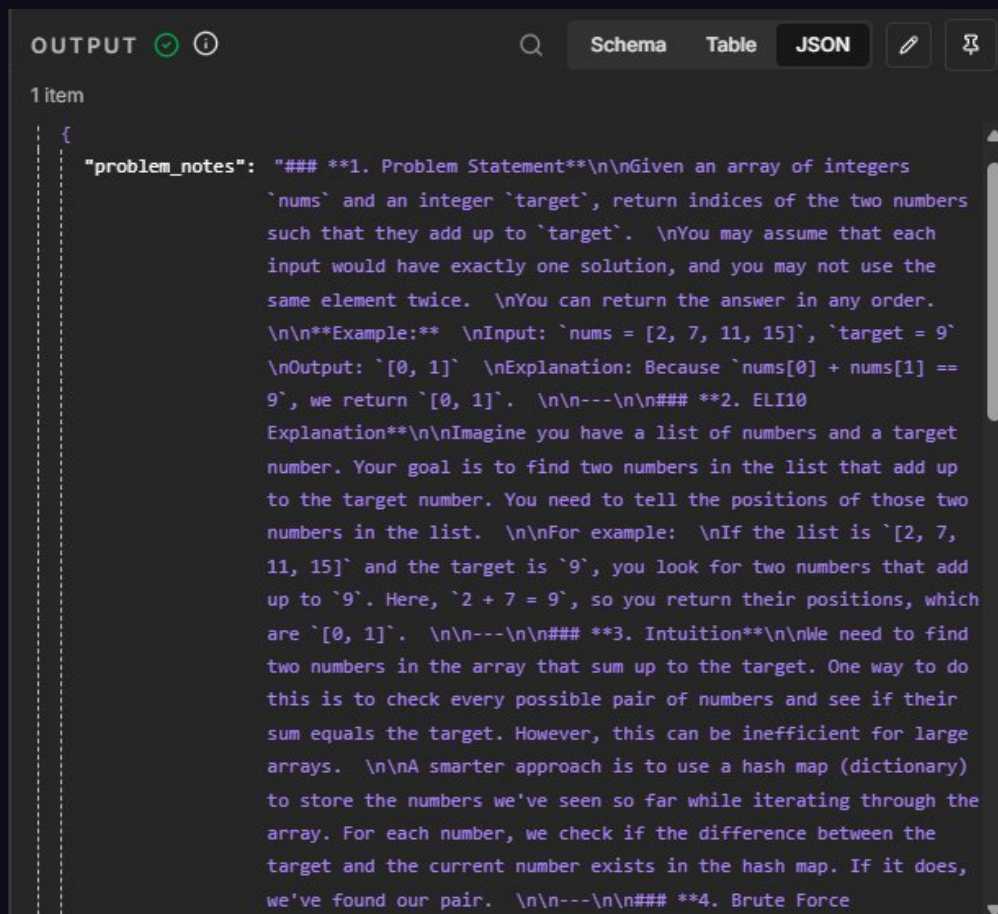


Figure 2 – SERPER API JSON output showing organic[0].title = 'Two Sum - LeetCode' and the direct problem link

6.3 OpenRouter / DeepSeek AI Output

The problem title extracted from SERPER is forwarded to OpenRouter, which routes the request to DeepSeek V3. The model returns a comprehensive JSON payload containing the problem_notes field with

full Markdown-formatted study notes.



The screenshot shows the n8n interface with the 'OUTPUT' tab selected. It displays a single JSON item with a field named 'problem_notes'. The value of this field is a multi-line string containing AI-generated study notes for the 'Two Sum' problem. The notes are formatted with Markdown, including section headers like '1. Problem Statement', '2. ELI10 Explanation', '3. Intuition', and '4. Brute Force'. The text is color-coded, with headings in purple and body text in light blue. The interface includes a search bar, tabs for 'Schema', 'Table', and 'JSON', and icons for editing and deleting the item.

```
{
  "problem_notes": "### **1. Problem Statement**\n\nGiven an array of integers\n`nums` and an integer `target`, return indices of the two numbers\nsuch that they add up to `target`. \nYou may assume that each\ninput would have exactly one solution, and you may not use the\nsame element twice. \nYou can return the answer in any order.\n\n**Example:** \nInput: `nums = [2, 7, 11, 15]`, `target = 9`\nOutput: `[0, 1]` \nExplanation: Because `nums[0] + nums[1] ==\n9`, we return `[0, 1]`. \n\n---\n\n### **2. ELI10\nExplanation**\n\nImagine you have a list of numbers and a target\nnumber. Your goal is to find two numbers in the list that add up\nto the target number. You need to tell the positions of those two\nnumbers in the list. \n\nFor example: \nIf the list is `[2, 7,\n11, 15]` and the target is `9`, you look for two numbers that add\nup to `9`. Here, `2 + 7 = 9`, so you return their positions, which\nare `[0, 1]`. \n\n---\n\n### **3. Intuition**\n\nWe need to find\ntwo numbers in the array that sum up to the target. One way to do\nthis is to check every possible pair of numbers and see if their\nsum equals the target. However, this can be inefficient for large\narrays. \n\nA smarter approach is to use a hash map (dictionary)\nto store the numbers we've seen so far while iterating through the\narray. For each number, we check if the difference between the\ntarget and the current number exists in the hash map. If it does,\nwe've found our pair. \n\n---\n\n### **4. Brute Force
```

Figure 3 – n8n JSON output panel showing the AI-generated problem_notes field with Problem Statement, ELI10 Explanation, Intuition, and Brute Force sections

6.4 Generated Markdown File

The final node converts the AI-generated notes into a plain-text Markdown file. The output panel confirms the file name (leetcode_1.md), extension (md), MIME type (text/plain), and file size (3.36 kB). The file can be viewed or downloaded directly from the n8n interface.

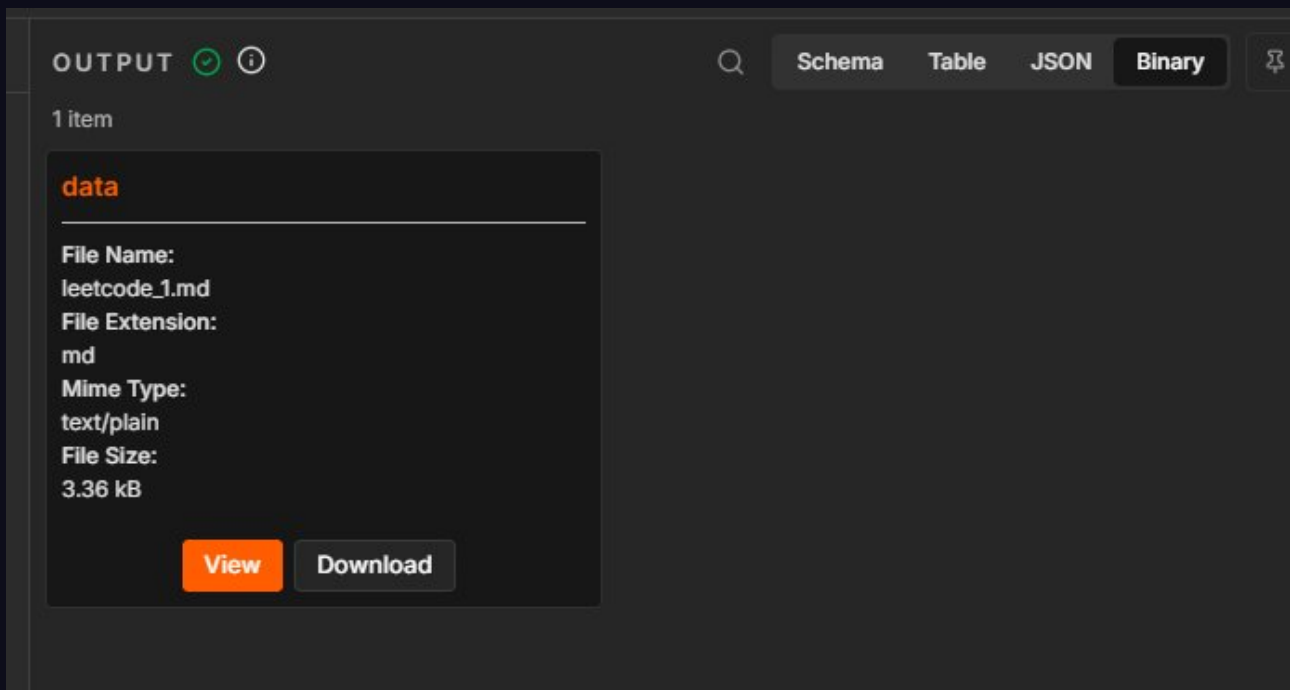


Figure 4 – n8n Binary output panel confirming leetcode_1.md (3.36 kB) was successfully created with View and Download options

■ 7. Sample Output

Input:

1

Detected Problem:

Two Sum - LeetCode

Generated study note sections:

1. Problem Statement

Given an array of integers `nums` and a target, return indices of the two numbers that add up to target. Each input has exactly one solution; the same element may not be used twice.

2. ELI10 Explanation

Imagine a list of numbers and a target. Find two numbers that add up to it and tell the positions of those numbers.

3. Intuition

Check every pair — or smarter: use a hash map to store seen numbers and check if the complement exists in $O(1)$.

4. Brute Force

Nested loops: $O(n^2)$ time, $O(1)$ space.

5. Optimal Approach

Single-pass hash map: $O(n)$ time, $O(n)$ space.

6 & 7. Complexity

Brute Force: $O(n^2)$ / $O(1)$. Optimal: $O(n)$ / $O(n)$.

8. Python Solution

```
def twoSum(nums, target): seen = {} for i, n in enumerate(nums): if target-n in seen: return [seen[target-n], i] seen[n] = i
```

9. Dry Run

`nums=[2,7,11,15]`, `target=9` → `i=0 n=2 complement=7` not in seen → `i=1 n=7 complement=2` in seen → return `[0,1]` ✓

■ 8. Features

- Automated LeetCode problem discovery via SERPER/Google Search
- AI-generated structured study notes with nine distinct sections
- ELI10 explanations for beginner-friendly understanding
- Both Brute Force and Optimal approach coverage
- Time and Space Complexity analysis for every solution
- Clean Python solution generation
- Step-by-step Dry Run examples
- One-click Markdown export for portable revision notes
- Fully automated end-to-end pipeline using n8n

■ 9. Future Enhancements

Enhancement	Description
Google Docs Export	Push generated notes directly to Google Docs
URL Support	Accept full LeetCode problem URLs as input
Multi-language	Generate solutions in Java, C++, JavaScript, and Go
Database Storage	Persist all generated notes in a PostgreSQL database
Web Interface	Build a React frontend for browsing and searching notes
Revision Dashboard	Personalised spaced-repetition dashboard for interview prep
Difficulty Filtering	Filter and batch-generate notes by problem difficulty

■■■ Author

Built by **Anuj** as a GenAI + Automation project combining n8n workflow automation, SERPER API for web search, and OpenRouter (DeepSeek V3) for AI-powered content generation. This project demonstrates practical integration of multiple APIs into a fully automated pipeline that produces revision-ready study notes for coding interview preparation.
